



Transcript

Week 11: Ensemble Modelling- Part 2

Video 11.1: Ensemble Modelling- Part 2 (01:54)

Let's revisit why relying on a single model often falls short. Real-world data is rarely clean or simple. As you know, ensemble models help by combining different perspectives—leading to more robust, data-driven decisions. Now, we'll build on that foundation to see how these models perform in practice.

Building on what we just revisited, ensemble methods stand out because they combine outputs from multiple models—each capturing different patterns in the data. This approach not only reduces overfitting and improves accuracy but also leads to more stable and reliable predictions across various scenarios.

As we saw earlier, ensemble models work by passing the same input data through multiple models—each offering a different prediction. These models may be similar or varied in type. The ensemble method then combines these outputs using techniques like averaging, voting, or stacking.

By revisiting this flow, we reinforce how ensemble models capture the strengths of individual learners to make more reliable predictions.

Now, think about a real-world scenario where combining predictions might outperform a single model. How might this approach benefit that case?

Video 11.2 - Advanced Ensemble Methods (01:33)

You already know that combining models can boost prediction accuracy—but what if you could take that even further?

Advanced ensemble methods like X-G Boost, Light G-B-M, and Cat Boost are powerful tools used by top data scientists to build faster, smarter, and more efficient models. These aren't just improved versions of basic techniques like bagging and boosting—they're complete game-changers. X-G Boost is known for its precision and ability to avoid overfitting through regularization. Light G-B-M stands out for its speed and performance with large datasets.

And Cat Boost excels at handling categorical features automatically. Together, these methods help you tackle complex machine learning problems with greater accuracy and confidence.

How could tools like X-G Boost, Light G-B-M, or Cat Boost sharpen your next machine learning project? Start exploring these game-changing techniques today.

Video 11.3 – X-G Boost (15:15)

Looking for a fast, accurate, and scalable machine learning solution? X-G Boost delivers top-tier performance with ease!

X-G Boost or Extreme Gradient Boosting is a high-performance machine learning framework designed for speed and accuracy. Developed by Tianqi Chen, it has gained recognition in competitions and real-world applications due to its efficiency. With parallel processing, X G Boost significantly reduces training time while advanced regularization techniques enhance model accuracy. Its scalability allows it to handle large datasets effectively, and built-in mechanisms prevent overfitting, ensuring reliable predictions.

X-G Boost is built on three core concepts: decision trees, ensemble learning, and the boosting process.

First, it uses decision trees as base learners. Each tree is trained step-by-step to minimize errors and improve predictions.

Second, it applies ensemble learning by combining the outputs of multiple trees. This collaboration increases overall accuracy.

Third, it follows a boosting approach, where each new tree focuses on correcting the mistakes made by the previous ones. This leads to progressively better results.

Together, these three ideas make X-G Boost powerful, accurate, and well-suited for handling complex data challenges.

X-G Boost stands out for its speed, accuracy, and adaptability. It uses both L-1 and L-2 regularisation techniques—this means it adds penalties for overly complex models to prevent overfitting, helping generalise better to new data. One of its strengths is handling missing values. X-G Boost automatically learns the best way to deal with gaps in data during training, so it stays reliable even when the dataset isn't perfect. It also supports parallel processing, which means it uses multiple CPU cores to train faster, especially with large datasets. Finally, its tree pruning method uses a depth-first approach to remove unnecessary branches, which results in simpler and more efficient models. Together, these features make X-G Boost a powerful tool for predictive tasks.

X-G Boost supports various loss functions for different tasks. For regression, it uses the formula shown to calculate the total loss. In this formula, N is the number of data points. Y_i is the actual outcome for a data point, and $\hat{y}_i$ is the predicted outcome. The function l measures the difference between the actual and predicted values, while Ω of f is the regularisation term that penalises model complexity.

Now, let's look at a specific example of a loss function: the squared error loss. This formula calculates the difference between the actual and predicted values for a given data point. It squares that difference to ensure all values are positive and to give more weight to larger errors. Here, y_i represents the actual outcome, and $\hat{y}_i$ is the predicted value for the same data point. The squared difference between them, written as open parenthesis y_i minus $\hat{y}_i$ close parenthesis squared, tells us how far off the prediction is.

This value is then used by the model to update and improve its predictions. The goal is to minimise this error across all data points, which helps the model become more accurate over time.

X-G Boost uses both L-1, also called Lasso, and L-2, known as Ridge, regularisation to improve model performance. These techniques add a penalty to the model's complexity, helping prevent overfitting. The formula shown combines both terms. It includes λ_1 multiplied by the sum of the absolute values of the weights, and λ_2 multiplied by the sum of the squared weights. This balance helps the model stay simple, generalise better, and perform well on new data.

Let's walk through a simple example of how X-G Boost works for a regression problem. Imagine we're trying to predict exam scores based on two features: the number of hours studied and the number of hours slept. We have two data points. In the first, a student studied for five hours and slept for six, resulting in a score of seventy-five. In the second, the student studied for three hours, slept for five, and scored fifty. This small setup helps us understand how X-G Boost uses features and outcomes to learn patterns in data.

In this step, we begin by making an initial prediction for all data points. Since we're working on a regression problem, X-G Boost starts by using the mean of the output values. Here, we take the average of seventy-five and fifty, which gives us an initial prediction of sixty-two point five.

Next, we calculate the residuals, which are the differences between the actual and predicted values. For the first data point, the residual is seventy-five minus sixty-two point five, which equals twelve point five. For the second data point, the residual is fifty minus sixty-two point

five, which equals negative twelve point five. These residuals will be used to train the first decision tree.

In Step 3, X-G Boost builds a decision tree to predict the residuals. It starts by calculating the gradient of the loss function for each data point. This gradient shows



how much the prediction needs to change. For Data Point 1, the gradient is positive twelve point five. For Data Point 2, it is negative twelve point five.

Next, the model looks for the best place to split the data. It calculates the gain for different split options. Gain tells us how much the prediction error is reduced when the data is split. In our case, the model checks splits based on $X-1$, which is hours studied, and finds that splitting at $X-1$ less than 4 gives the best result.

If a data point has $X-1$ less than 4, the model predicts negative twelve point five. If $X-1$ is greater than or equal to 4, it predicts positive twelve point five. These predictions help move the model closer to the true outcome.

In Step 4, we update the predictions using the learning rate, which controls how much the model adjusts based on each new tree. Let's assume the learning rate, also called η , is zero point one.

To update the prediction for each data point, we take the previous prediction—sixty-two point five—and add the learning rate multiplied by the output from the tree.

For Data Point 1, we calculate sixty-two point five plus zero point one times twelve point five, which equals sixty-three point seven five.

For Data Point 2, we do sixty-two point five plus zero point one times negative twelve point five, resulting in sixty-one point two five.

These updated values become the new predictions for the next step in the boosting process. In Step 5, we calculate new residuals using the updated predictions. For Data Point 1, we subtract the prediction of sixty-three point seven five from the actual value of seventy-five, giving us a residual of eleven point two five.

For Data Point 2, we subtract sixty-one point two five from fifty, which gives negative eleven point two five.

In Step 6, X G Boost uses these new residuals to build another decision tree. The model may choose to split on a different feature—such as $X-2$, hours slept—or it may refine the earlier split on $X-1$. This process repeats, with each new tree helping to reduce the remaining error.

In the final step, X-G Boost applies regularisation to help control model complexity. It uses two types: L-1, also called Lasso, and L-2, known as Ridge. L-1 regularisation penalises the absolute values of the weights, while L-2 penalises the squared values. These penalties are added to the loss function to prevent overfitting.

To get the final prediction, X-G Boost combines the outputs from all the trees. It multiplies their total by the learning rate and adds that to the initial prediction. This gives a balanced result— one that improves accuracy without making the model too complex.

To fine-tune an X-G Boost model, we adjust its hyperparameters. Key parameters include the learning rate, which controls how much the model updates in response to errors. A smaller learning rate improves accuracy but requires more trees.

Max depth sets how deep each tree can go—controlling complexity and reducing the chance of overfitting.

Subsample defines the fraction of data used to train each tree. Using only a part of the data helps prevent overfitting and makes the model more robust.

We also use optimisation techniques like grid search, which tests all combinations of values,

and random search, which picks values at random and often finds good results more quickly. Together, these settings help us get the best performance from the model.

X-G Boost is widely used across different machine learning tasks. For regression problems, it predicts continuous values—like house prices or stock performance.

For classification, it works well in both binary and multi-class cases, such as detecting



spam emails or classifying customer types.

It's also used in ranking problems, especially in recommendation engines and search result ordering.

X-G Boost handles time series forecasting too, by predicting future values based on historical data.

And finally, it's a favourite in Kaggle competitions, where data scientists rely on it for speed, accuracy, and robustness.

X-G Boost is a high-performance machine learning algorithm known for its speed and efficiency. With built-in L-1 and L-2 regularisation, it prevents overfitting and enhances model accuracy. Its flexibility allows support for multiple objective functions and evaluation metrics. The algorithm automatically handles missing values, eliminating the need for manual imputation. By leveraging parallel processing, it speeds up computation, making it ideal for large datasets. X-G Boost is also highly robust, effectively managing noise and outliers.

Additionally, it provides feature importance insights, aiding in better model interpretation and selection.

While X-G Boost delivers high performance, its complexity makes interpretation challenging compared to simpler models. Despite built-in regularization, improper tuning can lead to overfitting. Achieving optimal results requires careful hyperparameter tuning, such as adjusting the learning rate and number of trees. Though efficient, X-G Boost can be resource-intensive, especially with large datasets or deep trees. Additionally, while it can handle noise, excessive noise in the data may still reduce its effectiveness.

Now that you've explored the power and precision of X-G Boost, are you ready to apply it to your own data and see how far you can push predictive performance? Start experimenting, fine-tune your models, and bring your ideas to life with one of machine learning's most trusted tools.

Video 11.4 - Implementing X-G Boost in Python (06:37)

How do machines effortlessly recognise handwritten numbers? With X-G Boost, a powerful machine learning algorithm, we can train a model to accurately classify digits from the MNIST dataset. Let's explore how it's done!

We're building a digit classifier using X-G Boost and the popular M-N-I-S-T dataset. Each image is a 28-by-28 pixel grayscale picture of a handwritten digit. We'll flatten the image into a single row of 784 pixel values to use as input features. Our goal is to train X-G Boost to predict the correct digit label, from zero to nine.

This setup helps you see how X-G Boost handles image data efficiently, even when the feature space is large. We'll import essential libraries like X-G Boost, Num Pie, Pandas, and Mat plot lib. Then, we preprocess the input by flattening each image into a 784-feature array. The dataset is split into 60,000 training and 10,000 test samples, with the digit itself serving as the target variable.

To get started, we import essential Python libraries for data handling, visualisation, and model building. These include NumPy, Pandas, Matplotlib, Seaborn, and Scikit-learn, along with the X- G Boost library. We then load the M-N-I-S-T dataset using the fetch underscore

open M-L

function, and store the image data in a variable named capital X and the labels in lowercase y. Lastly, we check the shape of the features and target to confirm the dataset contains seventy thousand rows and seven hundred eighty-four features.

In this step, we're displaying random handwritten digit images from the dataset. To do this, we define a function called plot underscore random underscore images that takes images, labels, and an optional number of samples—defaulting to ten. We use Mat plot lib to create a two-row, five-column grid of images. Each image is reshaped to twenty-



eight by twenty-eight pixels and displayed in grayscale. The title above each image shows its corresponding label. This quick visual check helps confirm that our data is loaded correctly and gives us a sense of the variety in handwriting styles.

We begin by splitting the dataset into training and test sets using the train test split function. Here, the test size is set to zero point two, and the random state is set to forty-two for reproducibility. Next, we create an instance of the X-G Boost Classifier. We disable the label encoder and set the evaluation metric to log loss. Finally, we train the model using the fit function with the training features and labels.

We now use the trained model to make predictions. First, we predict the training data using the predict function and evaluate the results with a confusion matrix and classification report. Then, we do the same with the test data. This helps us understand how well the model performs on both seen and unseen data by comparing predicted and actual labels.

Now that we've trained the model, we evaluate its performance on the training data. The confusion matrix shows the number of correct and incorrect predictions for each digit. The classification report provides precision, recall, and F-one scores for every class. Here, all values are one point zero, indicating perfect predictions on the training set—which may suggest the model is overfitting.

Next, we assess how well the model performs on unseen data. The test confusion matrix shows a few misclassifications, while the classification report reveals high precision and recall across most classes. The average F-one score is zero point nine eight, indicating strong generalization with a slight drop in performance compared to the training results.

To improve our model's performance, we tune its hyperparameters using Grid Search. We define a parameter grid with values for the number of estimators, maximum depth, and learning rate. We then use the Grid Search C-V function to test different combinations and find the best settings based on accuracy. Finally, we print the best parameters and the highest cross-validation score.

After tuning the hyperparameters, we select the best model found by grid search. We use this model to predict the labels for the test set using the predict function. Finally, we print the predicted labels to verify that the model is working as expected.

Now that you've seen how to train, tune, and test an X-G Boost model, how would you apply this workflow to a different dataset? Try experimenting with new parameters and observe how they impact your model's accuracy.

Video 11.5 – Light G-B-M (06:10)

Did you know Light G-B-M can train faster and handle larger datasets than many traditional models? It's designed for speed, efficiency, and accuracy—especially when working with massive data. In the next few minutes, you'll discover how it works, where it fits best, and why it's a go-to tool for machine learning professionals. Let's dive into what makes Light G-B-M stand out.

Advanced ensemble methods build on basic ideas like bagging and boosting to create more accurate and reliable models. By combining different machine learning algorithms, they improve performance—especially on complex datasets.

Some of the most widely used ensemble methods include:

Gradient Boosting Machine, which improves predictions by correcting errors at each step; Light G-B-M, known for its speed and ability to handle large datasets and categorical features; X-G Boost, valued for its precision and built-in regularisation to prevent overfitting;

And Cat Boost, which handles categorical variables automatically, making it ideal for complex data.

Together, these methods expand what's possible in predictive modelling and help solve real-world machine learning challenges.

Light G-B-M is a high-performance gradient boosting framework built for speed and



efficiency. It introduces several innovations that set it apart from traditional methods like X-G Boost.

By using histogram-based algorithms, Light G-B-M approximates data distribution without relying on exact values, significantly reducing memory usage and computational cost.

It also implements exclusive feature grouping, which clusters similar features by frequency to minimise redundant splits during training.

Unlike traditional approaches, Light G-B-M directly optimises the loss function, eliminating the need for pre-sorting data.

Its support for parallel and distributed computing makes it well-suited for large datasets, while built-in handling of categorical features removes the need for one-hot encoding.

Additionally, Light G-B-M can leverage G-P-U acceleration for even faster training, especially in deep learning applications.

Together, these features make Light G-B-M a powerful choice for efficient and scalable machine learning.

Light G-B-M follows a step-by-step process to build accurate and efficient models. It starts by dividing the data into histogram bins based on feature values, which helps reduce memory usage and speeds up training. Then, it calculates gradients for each bin to determine how the model should adjust. Using these gradients, Light G-B-M finds the best split points in the data and constructs decision trees from those splits. Finally, it updates the model's parameters after each tree is added. This process helps Light G-B-M train quickly and handle large datasets effectively.

Light G-B-M supports a wide range of machine learning tasks across different domains. For classification, it's used to predict categorical outcomes—such as whether an email is spam or whether a customer is likely to churn.

In regression, it predicts continuous values, like estimating housing prices or forecasting future sales.

Light G-B-M is also effective in ranking problems, where items must be ordered based on their relevance—such as in search engines.

It plays a key role in recommendation systems too, suggesting items based on user preferences and behaviour.

Finally, Light G-B-M is useful for anomaly detection, identifying unusual patterns in the data—often applied in areas like fraud detection.

These use cases highlight Light G-B-M's flexibility and efficiency in handling both structured and unstructured problems.

Light G-B-M offers several advantages over X-G Boost, particularly for large datasets. Its histogram-based algorithm enables faster training and lower memory usage, making it more efficient. Unlike X-G Boost, Light G-B-M can directly handle categorical features without extra preprocessing, simplifying data preparation while preserving valuable information.

These features make Light G-B-M a strong choice for machine learning tasks requiring speed and efficiency.

Try replacing X-G Boost with Light G-B-M in one of your existing projects—compare the training time, memory usage, and accuracy. What do you notice?



Video 11.6 – Advanced XGBoost in Practice (14:21)

Welcome to this advanced hands on walkthrough of XG Boost, one of the most powerful and widely used machine learning libraries for gradient boosting.

But beyond performance, what sets XG Boost apart is its depth. It offers a wide range of configuration options and advanced capabilities that go far beyond simply calling the fit and the predict.

In this session we will use the California Housing Data Set, a real world data set ideal for regression, and walk through a complete modeling pipeline from loading the data, training and evaluating the model to using advanced features like monotonic constraints and custom metrics.

By end of this module you will not only be confident in using XG Boost for a standard modeling, but also in unlocking its true potential in high stakes real world applications.

Once again, as mentioned, Xgboost is one of the most powerful modeling technique that you have.

Let's start with our loading of the data.

We begin with one of the most crucial steps, understanding the data. We are using Fetch California Housing data set from Askelon which contains information about housing values and various predictors like income, house, age, number of rooms and geographical coordinates. This data set closely resembles what you would encounter in real world projects.

It is moderately sized, structured and rich in feature diversity.

We load the data into a data frame for ease of manipulation, and we explicitly name the target variable as target.

This setup allows us to inspect, transform, and nature model the data in a reproducible and structured way. Once the data is loaded, we prepare it for modeling.

We start by splitting the data set into training and testing using an 8020 split to ensure our model is evaluated on unseen data.

But unlike typical workflows, here we introduce D Matrix XG Boost optimized data structure.

Why are we doing that?

Because D Matrix is specially designed to improve efficiency. It stores data in a compact form, handles missing values internally, and accelerates training using advanced optimization techniques under the hood.

This is specially useful for large data sets or when training must be repeated many times during hyper parameter tuning.

Now that our data is in place, we initialize the AXG boost training.



Here we define basic training parameters.

Our objective is regression squared error, appropriate for regression problems, and we will monitor model performance using RMSE, A widely accepted error metric.

The seed ensures reproducibility and number of boost round sets the number of trees the model should build.

With the 100 trees we are establishing A baseline.

This model will serve as our starting point before we enhance and tune further. Think of this as laying the foundation before you start layering in complexity.

One of the XG Boost most effective tools for model regularization is early stopping.

This feature monitors the validation performance, in our case RMSE, on the test set and halts training when there is no improvement after a set of number of rounds.

This prevents the model from over fitting to the training data while also saving compute time.

Here we allow training up to 1000 rounds but stop early if no improvement is observed for 10 consecutive rounds.

The verbose output every 50 rounds help us track progress and understand the learning behaviour in real time.

To simplify it what we understand more number of iteration will reduce the error and my model will become more accurate.

There is also possibility that when I am increasing the number of boosting or number of iterations in a layman language, there is a possibility that my training error is going down but my test error after a certain iteration starts going up which shows over fitting.

What we are doing over here is we are watching for 10 consecutive rounds.

If the test errors keeps on going up then we are asking it to stop it so that my model doesn't over fit.

Before it over fits, we are stopping it.

That's a very good feature of Xgboost algorithm helps you to create more generalized models.

After training, it's essential to look under the hood and see what the model has learnt.

Xgboost Boost provides feature importance visualizations based on metrics like F score, gain and cover.

In our chart, medium income dominates some models decisions. This aligns well with domain expectations.

Higher income neighborhoods tend to have higher home values.

We also see geographical features like latitude and longitude playing important roles reflecting regional price variation.

These insights not only build trust in the model but also help in feature selection, communication, stakeholders, and future feature engineering.

Let's now measure how well our model performs.



We use RMSE to quantify the average error between predicted and actual values. Lower values mean better prediction.

We also calculate the R-squared score to understand how much variance in the target variable is explained by our model.

A high R square means our model is capturing the underlying patterns effectively.

These metrics help us compare across models and iterations.

While no model is perfect, these scores guide us in improving fit without over fitting. Once a model is trained and validated, the next step is often production or reuse in other pipelines.

Xgboost supports model persistence with save model and load model methods. This allows us to store our trained model as Jason file.

Easy to archive, deploy and Oregon integrate with API's or dashboards.

Loading it later ensures consistency and prediction without having to retrain saving both time and compute.

This code snippet will help you to save the model and if required you can call the model again.

Monotonic constraints applying domain logic.

Real world scenarios often involve domain knowledge that should be enforced in the model.

For example, it's logical that housing prices should not go down as income increases. XG Boost supports monotonic constraints to enforce such relationship. Here, we apply a constraint on the median income feature to ensure a monotonically increasing relationship with house prices.

This is specially useful in finance, healthcare, and policy domains where interpretability and fairness are critical.

With just a line of code, we make the model more trustworthy and aligned with human expectations.

If you look at the code over here, the value in one value of one in the third position enforces positive monotonicity for median income.

That's a third position over there in the given code.

And that's because in our given data, mid income, a median income is at the third position.

Off the shelf metrics don't always match business goals.

That's where we need custom evaluation function.



For example, in some cases under prediction is worse than over prediction. Or you may want to penalize large errors more severely. Here we write a custom root mean squared error function and pass it to using the `eval_metric` parameter.

This gives you full control over how model performance is evaluated during training.

Custom metrics can be crucial in industries like insurance fraud detection or logistics where the standard metrics fail to reflect true business cost.

Look at the code over here we are defining a custom root mean squared error which takes the predicted values and then `D.train`.

It gets a label from the `D.train` object and returns a custom root mean square error which is square root of predicted minus label whole square and taking mean of that.

Now in the model we pass, `eval_metric` is equal to custom RMSE to track and work with this custom metric that we have defined. Predict leaf indices.

This lesser known feature lets you see which leaf node a particular sample ends up in for each tree in the ensemble.

Each row in the output represents a sample, and each column corresponds to a specific tree.

Why is this useful?

Because leaf indices represent how the model segments the data, and these patterns can be used for customer segmentation, input to secondary model, or clustering.

It's a powerful tool for advanced analytics and explainability, bridging raw predictions with interpretable logic.

Think of them as some engineered feature which can be plugged in to a different model altogether.

That's another benefit you have with XG Boost models which is less known, but sometimes we can try this to build to work with more complex data.

To summarize, we have gone through an advanced feature rich workflow using XG Boost, from preparing data and establishing A baseline model to applying early stopping, visualizing feature importance and introducing advanced controls like monotonic constraints and custom matrix.

We also explored model explainability through leaf and dices. You now understand how XG Boost isn't just a performance oriented tool, it's a flexible framework for building interpretable and production ready models in real world settings.



Video 11.7 – XGBoost- An Intuitive Approach (19:30)

Welcome all to this session XG Boost and intuitive approach.

In this session I'm going to explain you the working of extreme gradient boosting with a different perspective.

Here if you look at our data we have area of the house given and price of the house given in 100,000 and the idea is to predict price from area the gradient boosting.

What it does is it starts with the initial prediction and based on the initial prediction it calculates errors and then then we model the error.

We build a model which can predict the errors and for that it uses a decision tree model.

That is why gradient boosting does and it uses generally A technique which is used in decision trees to split the branches and come up with the prediction.

An extreme gradient boosting will follow similar process, but how you build the tree is little different.

Now in this example we start with the initial prediction which you can take any value but generally what we do is whether it is regression or classification, we start with the initial prediction of 0.5.

So we have the same prediction for all the observations which is fine. This is my initial prediction.

You can take average of price as well as a initial prediction would not be a problem.

Next, what we do, we find the errors between the actual value and the initial prediction. So what is the difference between actual value and the initial prediction?

That is what you are getting over here. Most of them are +1 of them is negative.

Next, what we are going to do, we are going to choose our errors as our outcome variable Y and we are going to take area as X.

And we want to build a model which can predict our errors based on the input act. So we are not predicting the initial price, the actual price which has been given.

What we are predicting is error.

That's where your gradient boosting and exterior gradient boosting are different algorithm.

So we build a model, we want to build a model where we can predict the errors.

So what we want to do here, we want to distribute these errors into different branches and we want to see what is coming on a particular leaf.

So here these are my errors and we want to split into a tree structure and eventually want to give an output.

But how do we do this?

So we take these, we take this root node, we call it, this is our errors and we calculate something called quality score which is sum the errors and then take a square of it and divide it by number of error terms.

So in this example we are summing these errors and remember some of them are positive, some of them are negative, so they will cancel each other out to a certain extent and then we are squaring.

We are not first squaring and summing it up.

First we are summing it up and then we are squaring it and then we get this quality score.

We also add a regularisation parameter here, Lambda, which we will discuss later.



At the moment this is my quality score for these error terms and this we will call it as a root quality scores.

This is my root quality score.

So first step we have done how does it workout? I will explain you.

So if you see here we have the root quality score and now I want to build my tree which can predict errors.

So I am starting with area less than 1430. Why did I choose that?
I simply sorted the area in ascending order, pick up the 1st 2 value and find the average of that and that works as a cut off.

Same process that we follow in decision tree.

So here we are saying if area is less than 1/14/30.

If it is yes values are coming over here.

No values are going over here so we get 2 branches.

On one branches we have a single error term which is this.

On the other branch we have 4 values and then we calculate the quality score for each branch that we have.

So the left side branch would be sum the error terms on this particular leaf here and then square it and divide it by number of error terms which gives me this value here.

Then go to the right branch do the same thing, sum all the error terms, square it and divide it by total number of error terms.

That gives you this value which we are terming it as quality score.

And now what we do we calculate something called gain.

The gain is is nothing but what's the quality score on the left branch?

What's the quality score on the right branch and subtract the quality score for the root which we had over here created calculated when we are working with the errors and this is called the gain.

This is your gain when you are when you are taking area less than 1430 as a condition to split the tree, split the branch and create the tree.

So this is the gain that we have got.

Now what we will do here is we will calculate the gain for the next cut off which is taking these two as your area value, taking average of that and then taking that as a cut off splitting the branch and you get here 2 error term here and three of them will go here.

Calculate the quality score for each branch then left and right using the same concept. Sum of the error terms is squared divided by number of error terms and it gives me what's the quality score for the left and right branch when I am using 1765 as my cut



off.

And based on this I will calculate my gain in this scenario which comes to 0.16 and then I will try the next cut off and then the next cut off.
And whichever cut off gives me the maximum gain that we will be using as my final node to split the branches.

So that's how we decide how the tree should be splitted which is very different from what we have learnt in our decision tree process.

And this example area less than 11:40 will give you a better sorry area less than 14 odd amount will give you the best gain.

So we're gonna use that.

Now let's say using this concept, we have built a tree which looks like this area less than 1430 was the first condition.

We spread it the branch and then second level we have taken 1765 as the second cut off.

That's based on entirely on gain.

And at each level you have to keep answering this question that what's the gain overall and you have to split the tree.

And this is the final tree we have got here.

If you look at that, we have only one, we have a tree with with a single depth, but generally the depth will be kept at six.

That's a value that generally people use, so a tree with the depth of 6, but you are free to increase the depth value.

Now this is how my final tree has come up based on the quality score split criteria that I have spoken about.

Now how do we give the prediction?

If somebody says I have a house which is 2000 square foot, how will I give the prediction?

So the first thing it asks you is it less than 1430?

I say no, no.

Is this side it says is it less than 1765? I say no.

So I reach over here.

So this is the branch I have reached. So what's the error prediction?

Remember this is error prediction. So error prediction is given like this.

Some of the error terms on a leaf divided by number of error terms which is nothing but average of error terms.



And here again we will have the Lambda parameters.

But at the moment we are not using it. So that's how we will get the prediction.

Now here based on what I have mentioned to you, we gonna get the prediction. So this was the original data that we had area and price.

We start with the initial prediction. We calculate the errors.

I'm mentioning error one because we'll repeat this process N number of time. So we are saying here error 1 is this.

And then we build a model which will give me the predicted error and the predicted error in this case will come to this much.

If you reach over here, same value.

If you reach here, same value if you reach here, it's average of these three. And based on that we got the model prediction which is predicting errors. Remember that.

And now what we do is based on this first model that we have built will give you what will be the price prediction.

So price prediction would be what's your initial prediction which is this 0.5 and what was the models prediction for your error which was this sum these two up and you will get models price prediction.

So we are doing that.

So if you look at it, what I am saying is that my price prediction at the moment is equal to initial prediction plus your model prediction for the error, model prediction for the error.

That's what the prediction we have given here and now what we will do, we will calculate the error again.

I'm mentioning it as error 2. Second time I'm doing it. What's error 2?

What's your actual price and what's your Model 1 prediction for the price? Find the difference of that and that becomes error 2.

Interestingly, if you look at it, when we started, our error was this much for the first observation.

It became 0 means my predictions are 100% accurate and if you focus on all other errors, it has been reduced.

So my model is doing a better prediction.

Now if I start with the initial prediction and I go to the first prediction that we have done based on this concept, so error has been reduced.

Now what I'll do, I'll take this error as my Y and this area has my X and I will build another model which will predict my error too based on X using the same concept.

And once I get the predicted error, I'll get a new predicted price by adding this, the last model prediction for the price and the new error prediction that model is giving.

And I'll repeat this process. I'll keep doing it.



Artificial Intelligence and Machine Learning

I'll keep doing it till my error becomes 0 which generally doesn't happen in real time scenario.

So we'll keep doing this process until my error becomes very, very small.

And this number of iterations that we are running to predict errors is what we call as number of trees you are making and that's a parameter you an extreme they're in bursting.

So if you look at your prediction, the final prediction is nothing.

But you are saying what's your initial prediction plus your first model error prediction.

I'll say me one, treat it as model error, prediction one.

And then for the second model, this becomes the reference point and then we add me two model error 2 prediction and so on South, my actual price that comes at the end of say north number of trees that you are building would be equal to initial prediction plus your Model 1 prediction for the error, model 2 prediction for the error.

And I can say Minnesota, that's what you are doing and from each model you are becoming better and better.

That's my final prediction if I'm building N number of trees.

So this is what your extreme gradient boosting is doing.

Now when we are working with it, we also come across some hyperparameters. One of the hyperparameter is your ETA.

So what ETA does when I am saying that my price is equal to your initial prediction plus your first model prediction.

When I say first model understands the first tree we are building and we are predicting the errors.

So here this is the error prediction by the Model 1, then model 2 and then so on model N what learning rate does which is ETA it lets you control.

So here the learning rate will come and multiply with this it lets you control what proportion of this error term you want to use to adjust your initial prediction.

So if I say learning rate is 0.1, it means take all these individual predictions, sum that up and take 10% of that to adjust your initial prediction.

So it gives you a control in terms of how much contribution the trees will have towards your prediction.

This will help you to make your model less over fitted. That's one of the parameter. Another parameter is Lambda which is used when you are calculating the quality score.

When we were saying sum the errors square it divided by number of error term and then we add Lambda over there.

And it is also used when you are giving individual prediction which will be equal to sum of the errors divided by north plus Lambda.

Now Lambda is a regularisation term.

Changing Lambda will change the quality score which will change the way you are structuring your tree, which will eventually change your predictions as well.



Hence it gives you a control in terms of how to prune the trees.

So Lambda is another parameter that you can use. Another parameter is gamma. So gamma, what we do is we have been calculating the gain value and we set up a gamma value.

I can say gamma is equal to say 2.

And then we calculate gain minus gamma.

And if you recall, some of the gain values were 0.14. And if I say -2 which is less than 0.

If this calculated value of gain minus gamma is less than 0, then that particular branch will be pruned that will be removed.

So this will help you to remove unwanted branches from your model which might lead to overfitting.

So this is another parameter that you can use.

So you have hyperparameters, multiple hyperparameters in your extreme gradient boosting.

Some of them I have explained to you over here.

I hope this entire approach helps you to understand the model in a better manner and how it's done.

This was a regression problem for the classification. The overall concept is same. But when we when we are coming with the initial prediction, we come up with the initial prediction of the log likelihood ratio that we have and then convert them into probability scores and the model predicts a log likelihood and we work with that. But overall concept is same.

Also in the classification problem when we are giving the prediction, it has a different formula altogether but the concept is same overall.

So I hope this is helpful to you to understand how extreme gradient boosting works.

Thank you.